

Rec4LLM: LLM-Based Recommendation System with Soft & Hard Prompting

Edition: Jin Yuze 17 May 2025

Overview and High-Level Goals

Rec4LLM (based on the [CLLM4Rec](#) framework) is a large language model (LLM)-based recommendation system that tightly integrates traditional ID-based collaborative filtering with rich textual content understanding.

The key idea is to extend a pretrained [GPT-2](#) model with user and item ID tokens so that the model can learn user–item interaction patterns (the “ID paradigm”) while also leveraging textual data like item descriptions or user reviews (the “LLM paradigm”). By combining these, Rec4LLM aims to address challenges such as shallow understanding of content and poor generalization in classic recommenders.

Goals of Rec4LLM:

- **Unified representation:** Represent users and items as special tokens within the GPT-2 vocabulary. This allows the model to embed IDs in the same semantic space as words, capturing intrinsic user interests and item properties.
- **Soft + hard prompting:** Develop a prompting strategy where “soft” tokens (user/item IDs) are used alongside “hard” tokens (natural language) in prompts. This means the model sees prompts that include both ID tokens and descriptive text, helping it connect collaborative signals with content.
- **Mutual regularization:** Train the model on two types of data (collaborative interactions and content) in a way that the learned user/item embeddings are consistent across both. A regularization term ties the two learning objectives together, encouraging the model to capture recommendation-relevant information from noisy text data.
- **Recommendation-oriented fine-tuning:** After pretraining, further fine-tune the model specifically for recommendation tasks. This involves adding an item prediction head so the model can directly output a set of recommended items given a user’s history. The fine-tuning is designed to generate multiple item recommendations efficiently without the model “hallucinating” invalid or irrelevant items.

In summary, Rec4LLM’s architecture uses GPT-2 as a backbone and treats recommending items as a language modeling problem – but with special tokens and training techniques that make it specialized for personalized recommendations. The next sections will walk through each component of the project (data processing, model structure, training loops, etc.) and explain how they implement these ideas.

Architecture and Key Concepts

Extending GPT-2 base model with User and Item Tokens

Rec4LLM uses GPT-2, a "transformers model pretrained on a very large corpus of English data in a self-supervised fashion", as the base. It extends the model's vocabulary to include unique tokens for each user and each item in the dataset. In the [paper](#), the authors detailedly explained why they need to perform this extension. Briefly speaking, this is to ensure that the model can process user and item IDs correctly, rather than breaking them into subword tokens, which would lose the semantic meaning of the IDs.

In the implementation, this is achieved by creating new embedding matrices for users and items and integrating them with GPT-2's existing word embeddings:

```
# in model.py (GPT4RecommendationBaseModel.__init__)
# Create new token embeddings for user/item tokens
self.user_embeddings = nn.Embedding(self.num_users, config.n_embd)
self.item_embeddings = nn.Embedding(self.num_items, config.n_embd)
# Randomly initialize the new token embeddings
self.user_embeddings.weight.data.normal_(mean=0.0, std=config.initializer_range)
self.item_embeddings.weight.data.normal_(mean=0.0, std=config.initializer_range)
```

In the code above, `config.n_embd` is the dimension of GPT-2's embeddings, so user and item IDs get the same embedding size as normal vocabulary tokens. These embeddings are learned parameters that will encode collaborative information (for user IDs) and item properties (for item IDs).

User and item IDs as special tokens: The user and item tokens are not part of GPT-2's original vocabulary. Instead, they are indexed in a separate range. For example, if the original GPT-2 vocabulary size is V (around 50k for GPT-2), then an input token $ID < V$ is treated as a normal word piece, an ID in the range $[V, V+U)$ is a user token, and an ID in $[V+U, V+U+I)$ is an item token (where U is number of users, I is number of items). The model's embedding lookup uses these ranges to select the appropriate embedding:

```

# model.py (GPT4RecommendationBaseModel.embed)
vocab_mask = (input_ids < self.vocab_size).long()
user_mask = ((input_ids >= self.vocab_size) &
              (input_ids < self.vocab_size + self.num_users)).long()
item_mask = (input_ids >= self.vocab_size + self.num_users).long()

# Use GPT-2 embeddings for vocab tokens
vocab_ids = (input_ids * vocab_mask).clamp_(0, self.vocab_size - 1)
vocab_embeddings = self.gpt2model.wte(vocab_ids) * vocab_mask.unsqueeze(-1)

# Use new embedding matrices for user and item tokens
user_ids = ((input_ids - self.vocab_size) * user_mask).clamp_(0, self.num_users - 1)
user_embeddings = self.user_embeddings(user_ids) * user_mask.unsqueeze(-1)

item_ids = ((input_ids - self.vocab_size - self.num_users) * item_mask).clamp_(0, self.num_i
item_embeddings = self.item_embeddings(item_ids) * item_mask.unsqueeze(-1)

# Sum up embeddings for all token types
input_embeddings = vocab_embeddings + user_embeddings + item_embeddings

```

This code fragment shows how any input sequence is embedded. The model creates boolean masks to detect which tokens are regular words, which are user IDs, and which are item IDs. It then looks up each token in the appropriate embedding space and sums the vectors. Summing works because for each position, only one of the masks will be 1 (and the others 0), so effectively it selects the correct embedding. The summed input_embeddings are then passed into GPT-2's transformer layers. This mechanism cleanly aligns user/item tokens to the LLM's representational space.

Importantly, the GPT-2 model itself is also slightly modified to accept an extra input: `inputs_graph_bc`. We will touch on this later – it's a matrix encoding the bi-directional connections (BC) between user and item tokens (essentially a pre-computed graph of user-item interactions). The base model's forward method passes this to the underlying transformer (`self.gpt2model`) to inform its attention layers:

```

# model.py (GPT4RecommendationBaseModel.forward)
return self.gpt2model(inputs_embeds=input_embeddings, inputs_graph_bc=mapping_graph_bc, **kw

```

(The `mapping_graph_bc` provides the model with knowledge of which user-item pairs have interacted, adding an inductive bias to the attention mechanism. We won't deep-dive into the transformer implementation, but conceptually it helps the model attend more strongly between related user and item tokens.)

Soft + Hard Prompting Strategy

What is soft vs hard prompting? In this context, “soft tokens” refer to the learned ID tokens (*user_id* and *item_id*), which do not carry intrinsic semantic meaning to humans but represent entities in the data. “Hard tokens” refer to ordinary words or phrases in natural language. Rec4LLM uses a combination of both in its input prompt.

- The prompt part of each input sequence contains a mix of user/item tokens (soft) and descriptive language (hard).
- The main text part of each sequence is more homogeneous – it’s either all item tokens or all natural language tokens, depending on the task, to make the language modeling objective more stable.

This design is best explained by example. During **pretraining**, the project constructs two kinds of documents:

1. **Collaborative (Interaction) prompts:** These are synthetic sentences describing a user’s interaction history. For example, a prompt might read:

`“user_42 has interacted with item_5 item_17 item_36, user_42 will interact with”`

Here, `user_42` , `item_5` , etc. are soft tokens, while the connecting phrase “has interacted with” and the clause “will interact with” are hard tokens (normal English words). The prompt ends with “will interact with” , inviting the model to predict which item comes next. The main text to predict in this case would be one or more item tokens – representing items that the user is expected to interact with in the future (hold-out items). By construction, this main part consists only of item IDs (no natural words), i.e., a homogeneous sequence of item tokens.

In the code, such a prompt is generated in `data.py` . For each user `idx`, the code gathers their past interacted items and creates a string as shown above. For example:

```
input_prompt = f"user_{idx} has interacted with " \
    + " ".join([f"item_{item_id}" for item_id in input_interactions]) \
    + f", user_{idx} will interact with"
```

This corresponds to the structure described, inserting the user and item IDs appropriately. (If a portion of interactions are masked for prediction, those masked ones are simply omitted from the prompt string.)

2. **Content (Review/Description prompts):** These incorporate actual text about items or from users. For example, a prompt could be:

```
"user_42 writes the following review for item_36:"
```

followed by a piece of text like "This item is too expensive." which is the main text the model should produce. In this prompt, `user_42` and `item_36` are soft tokens, while the phrase "writes the following review for" and the punctuation are hard tokens. The main text is a natural language sentence containing no special ID tokens (only regular vocabulary).

The code preparing such data (`UserItemContentGPTDatasetBatch` in `data.py`) loads pairs of prompt text and main text. Each pair is something like:

```
("user_1 writes the following review for item_1:", "This item is too expensive.")
```

as illustrated in the code comments. The prompt portion mixes the ID with a natural-language template, and the main portion is the actual review text the model must generate.

Why soft+hard prompting? According to the CLLM4Rec paper, splitting the input this way helps stabilize training and ensures the model learns effectively from heterogeneous information. The prompt provides the model with both the collaborative identifiers and some linguistic context (so it knows how to "interpret" those IDs in a sentence), while the main text is kept uniform (all items or all words) to make the language modeling task well-defined. In other words, the model isn't asked to unpredictably switch between generating an English word and an item ID at the next step – that could confuse it. Instead, each training instance is set up so that after the prompt, the model generates either a sequence of item tokens or a sequence of normal word tokens. This approach is a novel solution proposed in CLLM4Rec to leverage "heterogeneous tokens" in the prompt but still have a clear generative target.

Mutual Regularization between Collaborative and Content Learning

A core challenge is ensuring that the user/item embeddings learned from pure interaction data (IDs only) and those learned from content data (IDs in textual context) are aligned. Rec4LLM addresses this with a mutual regularization strategy. In practice, this means when training on one type of data, the model tries to keep the representations of the user/item tokens close to what was learned from the other type of data.

Concretely, during training the code computes an additional loss term that measures the difference between the embeddings produced in the collaborative scenario and those from the content scenario. For example, when the model is training on a batch of interaction data, it will:

- Take the sequence of tokens (user + item IDs in prompt, item IDs in main) and embed them via the base model's `embed()` function, yielding a sequence of embedding vectors (one per token position).
- Separately, take the **same sequence of IDs** (or the user and item IDs in that sequence) and feed them to the content-based model's embedding function (which has seen text data) to get another set of

embedding vectors.

- Compute a loss (via Mean Squared Error) between these two sets of embeddings.
- Add this *regularization loss* to the main training loss, with a weighting factor `lambda_V` (configurable).

The effect is to **encourage the collaborative model's embedding space to stay close to the content model's embedding space** (and vice versa). Intuitively, if the content-based model has learned that `item_36` 's embedding should align with the meaning of the word “expensive” (because many reviews for `item_36` mention “expensive”), the collaborative model is nudged to also embed `item_36` in that region so that it doesn't contradict the content understanding. This cross-pollination makes the learned representations more robust and semantically meaningful.

In the code, mutual regularization is toggled by a flag `regularize=True` and executed in the model forward passes. For example, in the collaborative pretraining model (`CollaborativeGPTwithItemLMHeadBatch`), we see:

```
if regularize:
    # Combine prompt and main embeddings from the collaborative model
    collaborative_embeds = torch.cat(
        (self.base_model.embed(input_ids_prompt),
         self.base_model.embed(input_ids_main)), axis=1
    )
    # Compute MSE loss against content embeddings passed in
    regularize_loss = lambda_V * torch.mean(
        nn.MSELoss(reduction='sum')(collaborative_embeds, content_embeds)
    )
    loss += regularize_loss
    outputs = (loss, regularize_loss) + outputs
```

And similarly, the content model will add a regularization loss if `regularize=True` (comparing its prompt embeddings to `collaborative_embeds` passed in). In practice, during the actual pretraining run, the implementation first trains the content model alone (no regularization in that phase), then during collaborative training it freezes the content model and uses these losses to align with the already learned content embeddings (more on the training order in a moment).

Recommendation-Oriented Fine-Tuning with an Item Prediction Head

After the pretraining phase, Rec4LLM performs a special **fine-tuning** stage geared specifically towards making recommendations. The idea is to use the pretrained model as a backbone and add a lightweight item prediction head on top, then train the model to predict held-out items from a user's history.

Item prediction head: This is essentially a linear layer that maps the model's hidden state to a distribution over the item vocabulary. In the code it's defined as

`item_head = nn.Linear(n_embd, num_items, bias=False)` and then **tied to the item embedding matrix**. Weight tying means the linear layer's weight matrix is actually the transpose of the item embedding matrix. This way, generating an item uses the same representations as encoding an item, a technique known to improve consistency in language models (similar to tying word embeddings with output softmax weights). As a result, the logits produced by this head effectively compute a dot product between the model's hidden state and each item's embedding vector.

During fine-tuning, the process is:

- The model reads a prompt that includes the user ID and some of their interaction history (with some interactions masked out). The prompt is structured with soft+hard tokens as before (e.g. "user_42 has interacted with item_5, item_17, ..., user_42 will interact with"). The masked-out interactions are omitted from the prompt even though we know the user did interact with those items – these will serve as the **ground truth items to predict**.
- The input sequence (prompt) is fed through GPT-2 (with user/item tokens embedded as learned). Notably, in fine-tuning we often provide the entire prompt as input and do **not** provide the item tokens that were held out (the model needs to predict those, not just copy them).
- Instead of generating tokens one by one autoregressively, the model uses the item prediction head on the **final hidden state of the sequence** to directly produce a probability distribution over all items. Essentially, GPT-2 processes "... will interact with" and then we look at the transformer's output at that position (where generation would happen next). This single transformer forward pass can be used to score every item at once, which is much more efficient than sequential generation.
- We train the model with a **multinomial likelihood loss**: we want the probability mass to be concentrated on the actual items the user will interact with (the masked ones). If a user had (say) 3 hidden interactions, the loss is the negative log-likelihood of those 3 items' IDs under the predicted distribution. In code, if `item_log_probs` is the log-softmax output from the item head and `target_ids` is a one-hot vector for the true items, the loss is computed as $-1 * \text{sum}(\text{item_log_probs} * \text{target_ids})$. This effectively adds the log probabilities of the correct items (since for those entries `target_ids = 1`) and negates them (so it's high when the model is wrong, and zero if the model assigns probability 1 to all correct items). The model is trained to minimize this negative log-likelihood.
- During training, the implementation actually supplies `target_ids` (also called `target_matrix` in code, a multi-hot vector of the holdout items) so it can compute the loss in a vectorized way. But crucially, the target items are **not fed as input tokens** in the prompt; they are only used for the loss. This is how the model learns to predict them.
- The fine-tuning routine continues to include the regularization term with the content model's embeddings (the content model remains available from pretraining). This ensures the user/item

embeddings don't drift too far from the semantic space while we fine-tune on pure recommendation loss.

By the end of fine-tuning, we have a model that, given a user and some history, can output a ranked list of items. The approach of using the item prediction head means the model can recommend multiple items in one shot by taking the top- K probabilities from that output distribution. This avoids the need for the model to generate items one-by-one (which would be an auto-regressive process and could lead to repeats or “hallucinated” outputs). The paper highlights this as a major advantage: *“recommendations of multiple items can be generated efficiently without hallucination.”*

Code Modules and Their Roles

Now, let's connect the concepts above to the actual code structure of the Rec4LLM project. The project is organized into a few key Python modules:

- `libs/data.py` – Data loading and prompt generation logic. Defines how user-item interactions and content data are turned into the input format for the model.
- `libs/model.py` – Model architecture definitions. Contains classes for the base GPT-2 with extended embeddings, as well as the specialized model classes for content-based training, collaborative training, and recommendation head.
- `llm4rec_training.py` – The main script for the pretraining stage. It sets up the dataset and model objects, then runs the two-phase pretraining: first on content data, then on interaction data with mutual regularization.
- `llm4rec_finetuning.py` – The script for the fine-tuning stage. It loads the pretrained models, adds the recommendation head, then trains on a hold-out prediction task and evaluates the recommendation performance (Recall, NDCG metrics).

Concise Walkthrough of Code Modules

Data Preparation (`data.py`)

This module defines dataset classes to produce the prompt-main sequences for both content and collaborative data.

- **Content data class (`UserItemContentGPTDatasetBatch`):**

This class loads pairs of prompt text and main text from preprocessed files.

Each data point corresponds to a user-item pair with associated text. The `__getitem__` simply returns these two strings.

The `collate_fn` for this dataset is important: it takes a batch of such (prompt, main) pairs and encodes them with the GPT-2 tokenizer. It returns `prompt_ids`, `main_ids`, and an `attention_mask` that indicates which positions are real tokens (1) vs padding (0).

One subtlety is how the attention mask is constructed: the code concatenates the prompt and main sequences and makes sure to properly pad/truncate so that the total length doesn't exceed the model's maximum (1024 tokens).

It likely also constructs `mapping_graph_bc_prompt` and `mapping_graph_bc_combined` here (not shown in the snippet) by calling `get_bc_from_mapping`.

For content, `mapping_graph_bc_prompt` would encode connections among user/item tokens in the prompt, and `mapping_graph_bc_combined` might do so for the prompt vs the combined sequence. However, since the main text in content data has no user/item tokens (only vocab tokens), the graph for combined may effectively just cover prompt tokens as well.

In any case, these matrices are passed along to inform the model's transformer layers about user-item relationships present in the prompt.

- **Collaborative data classes:**

There are two related classes for the interaction data: one for training

(`RecommendationGPTTrainGeneratorBatch`) and one for validation/testing

(`RecommendationGPTTestGeneratorBatch`).

They handle the difference between fine-tuning on partially observed histories v.s. evaluating on a full history.

- For fine-tuning training: The training generator takes a sparse user-item interaction matrix (`train_mat`) and will on the fly mask a fraction of each user's interactions as targets.
- For validation/testing: The test generator is used to evaluate recommendation performance after fine-tuning. It uses `train_mat` (the training interactions for each user) as the known history and `val_mat` (or `test_mat`) as the ground truth future interactions.

In both cases, `mapping_graph_bc` (a large matrix loaded from file) is used to fill in the `graph_bc` tensor. This matrix contains precomputed connections between any two IDs (`user-user` , `user-item` , etc.). Practically, it's an adjacency matrix of the user-item bipartite graph (and possibly additional content-similarity edges) that the model can use as prior knowledge.

The `get_bc_from_mapping` function sets `inputs_graph_bc[i,j] = 1` if both positions `i` and `j` in the sequence are ID tokens and the corresponding IDs have a connection in `mapping_graph_bc`. For example, if position `i` is `user_42` and position `j` is `item_36` and user 42 interacted with item 36, then `inputs_graph_bc[i,j] = 1`.

These matrices are fed to the model's transformer to bias its self-attention – effectively informing the model which tokens (users/items) are known to be related in the data. This is an advanced feature that injects collaborative graph knowledge directly into the sequence modeling process.

Model Structure (`model.py`)

As discussed earlier, `model.py` defines several classes to implement the Rec4LLM architecture:

- **GPT4RecommendationBaseModel**

The base network that wraps a GPT-2 Transformer (GPT2Model) and adds user/item embeddings. It overrides how inputs are embedded (using the `embed()` method we saw) and simply forwards the resulting embeddings into the GPT-2 model. This base is used by all the following specific model classes. Notably, GPT-2's config is extended with `num_users` and `num_items` attributes to inform the sizes of the new embeddings. The code uses a custom `GPT2ModelWithBC` class when instantiating GPT-2; this is to accept the `inputs_graph_bc` tensor in its forward pass (so that the self-attention can incorporate those biases).

- **ContentGPTForUserItemWithLMHeadBatch**

This model is used for the content-based pretraining stage. It comprises a `base_model` (`GPT4RecommendationBaseModel`) and an LM head for language modeling.

(A very detailed explanation of the components, which is optional to read, is attached to the end of this document)

- **CollaborativeGPTwithItemLMHeadBatch**

This model is used for the collaborative pretraining stage. It is structurally similar to the content model but uses an item prediction head instead of a vocabulary LM head.

(A very detailed explanation of the components, which is optional to read, is attached to the end of this document)

- **CollaborativeGPTwithItemRecommendHead**

This is the model used for fine-tuning and evaluation.

It also has an `item_head` tied to item embeddings, but its usage is different from the above.

Instead of doing language modeling over a sequence of items, it is designed to output one set of item scores for an entire prompt.

(A very detailed explanation of the components, which is optional to read, is attached to the end of this document)

Pretraining (`llm4rec_training.py`)

This training script orchestrates the two-stage pretraining: (1) content-based and (2) collaborative-based.

Key steps in `llm4rec_training.py` :

- **Initialization:**

It loads the preprocessed data and builds dataset objects. For example, it loads a list of content file paths and creates

```
content_data_gen = UserItemContentGPTDatasetBatch(tokenizer, file_list, mapping_graph_bc)
```

. It also loads the interaction matrices and creates `review_data_gen` (possibly alias for content) and `collaborative_data_gen` .

- **Model setup:**

The script prepares the model config by extending GPT-2's config with `num_users` and `num_items` . It then instantiates two separate GPT-2 based models:

- The content model: create a GPT-2 (with BC) instance and wrap it in `GPT4RecommendationBaseModel`. Then create `ContentGPTForUserItemWithLMHeadBatch` with that base.
- The collaborative model: similarly, create another GPT-2 instance, wrap it, and then `CollaborativeGPTwithItemLMHeadBatch`.

These two models (`content_model` and `collaborative_model`) are separate but initialized from the same GPT-2 weights (assuming both `GPT2ModelWithBC` instances load the pretrained GPT-2 parameters) so that they start from a common point. The user/item embedding matrices in each are initialized randomly as per code. Essentially, at the very start, the two models have identical transformer weights but different initial embeddings for the new tokens.

- **Pretraining Stage 1 – Content-based:**

The script trains `content_model` first, while `collaborative_model` is untouched.

Similar to regular ML model training process, it sets up an optimizer for the content model's parameters and loops for a specified number of epochs.

By the end of it, `content_model` should have learned meaningful representations for user and item tokens in context of text.

- **Pretraining Stage 2 – Collaborative-based:**

After content pretraining finishes, the script moves on to collaborative pretraining. Now it will train `collaborative_model`, but importantly it will **use the `content_model`'s knowledge for regularization**.

The content model can be either fixed (as a teacher) or potentially still trainable, but in practice the code uses `with torch.no_grad(): content_embeds = ... content_model.embed(...)` during collaborative training, indicating it's treating the content model as fixed. (The content model's parameters are not updated in this second stage; it's just providing target embeddings.)

By the end of this stage, `collaborative_model` has learned to model interaction sequences while staying aligned with the content model's embedding space.

Note: The pretraining procedure as implemented is **sequential**: first content, then collaborative. The paper conceptually describes mutual reg as if both learn together, but in practice, the content model is taught first (on text) and then the collaborative model learns with guidance from the frozen content model. This is a design choice likely made for implementation simplicity. It still achieves the goal that at the end of pretraining, we have two aligned models. (One could also alternate training steps between them, but that's not done here.)

Fine-Tuning (`llm4rec_finetuning.py`)

This script takes the pretrained models and performs the final fine-tuning to directly optimize recommendation accuracy.

Main steps in `llm4rec_finetuning.py` :

- **Loading pretrained models:**

It reads the saved checkpoint files. The code creates a GPT-2 config (with num_users/num_items) same as before. Then it instantiates `content_model` and `collaborate_model`. This sets up the model with the recommendation head on top (the `item_head` is created and tied to item embeddings inside the class's `__init__`).

- **Recommendation-oriented Fine-tuning:**

Optimizes the model with masked user-item prompts to predict held-out items directly, using a dedicated recommendation head.

Load `train_mat` and `val_mat`, create: `collaborative_train_data_gen` (this will yield masked prompts and targets as describe) and `collaborative_val_data_gen` (yields full prompts and separate known/target matrices for evaluation).

- **Evaluation:**

Periodically evaluates the model's recommendation accuracy (using **Recall** and **NDCG** metrics) on a validation set, ensuring the learned embeddings lead to meaningful recommendations.

At the end of fine-tuning, we expect the model to have learned to make accurate recommendations. The final model (`collaborative_model` with recommend head) can generate a ranked list of items for any user prompt by a single forward pass.

Connecting to the CLLM4Rec Architecture

To ensure clarity, let's map the pieces we've discussed to the terms introduced in the CLLM4Rec paper:

- **“Extending LLM vocabulary with user/item ID tokens”** – Implemented via the additional embedding matrices and the integrated `embed()` function in `GPT4RecommendationBaseModel`. This is how Rec4LLM *“faithfully models user/item collaborative and content semantics”* as stated in the paper.
- **“Soft+hard prompting strategy”** – The way prompts and main texts are constructed in Rec4LLM directly reflects this. Each training “document” has a prompt comprised of **heterogeneous tokens** (user and item IDs = soft tokens, plus connecting words = hard tokens) and a main part of **homogeneous tokens** (either all item IDs or all vocabulary tokens). We saw this in the data generation for both interaction sequences and textual reviews. This design facilitates *“stable and effective language modeling”* by not mixing token types in the predictive part of the sequence.
- **“Mutual regularization”** – In code, this is the MSE loss aligning content and collaborative embeddings. The paper describes it as a strategy to *“encourage CLLM4Rec to capture recommendation-related information from noisy user/item content”*. By training the ID embeddings on content (which can be noisy or verbose) and then regularizing collaborative training with those embeddings, the model is guided to not ignore content info and simultaneously to not overfit just to ID co-occurrences.
- **“Recommendation-oriented finetuning”** – This is exactly the second stage where we add the item prediction head with multinomial likelihood. Instead of continuing to treat recommendation as a pure

sequence generation, the fine-tuning optimizes the model to directly predict held-out items from a masked history prompt. The item head approach ensures “*recommendations for multiple items can be generated efficiently without hallucination*” – the model’s outputs are constrained to real item IDs and it can output a set of them in one go. The evaluation metrics (Recall@K, NDCG@K) are directly computed from these outputs, aligning the training objective with standard recommender benchmarks.

- **Use of both collaborative and content information** - The overall architecture tightly couples the two: user/item IDs carry collaborative signals, but they are trained in tandem with content (text) such that the final recommender has a richer understanding of who the users are and what the items are. In effect, Rec4LLM can be seen as a “collaborative large language model” for recommender systems – hence CLLM4Rec. It merges the strengths of ID-based matrix factorization (personalized latent factors) with those of language models (semantic generalization).

Conclusion

Rec4LLM is a comprehensive example of an LLM-based recommender system that implements state-of-the-art ideas from recent research. We walked through how the code builds prompts (soft+hard), how it modifies GPT-2 to accept new tokens, and how it trains in two stages to incorporate both textual and interaction data.

For a BT4222 student, key takeaways include:

- **Representation learning with IDs in NLP models:** Even though user IDs and item IDs are not natural language, we can plug them into a language model and train embeddings for them, effectively learning “meaning” for these IDs based on usage (interactions or contexts in text).
- **Prompt design matters:** The way we feed data to an LLM (ordering of tokens, mix of words and IDs) can significantly affect learning. The soft+hard prompt approach is a clever way to blend structured data into an LLM’s text input.
- **Multi-task training and regularization:** Rec4LLM’s pretraining is a multi-task learning problem (next-word and next-item prediction). The mutual regularization loss acts as a bridge between tasks, a form of “model alignment” ensuring that what is learned from one task is not lost in the other.
- **Adapting generation for recommendations:** Language models typically generate one token at a time, but recommending a set of items required a tweak – the item recommendation head – to make the output a set of classes (items) rather than a fluent sentence. This highlights how we sometimes need to depart from pure language modeling when applying LLMs to other domains.

By understanding the theoretical foundations (from the CLLM4Rec paper) and stepping through the implementation logic, we can appreciate how Rec4LLM brings together collaborative filtering and natural language processing in a unified model. This opens up many possibilities – e.g., the same model could

potentially explain its recommendations (using its language ability) or handle new items/users via descriptions. It's a compelling direction at the intersection of recommender systems and NLP.

Sources

- [CLLM4Rec: A Large Language Model for Recommendation with Soft and Hard Prompting](#)
- [Rec4LLM GitHub Repository](#)

Appendix

Optional reading material for explanations on the model details.

Detailed explanation of `ContentGPTForUserItemWithLMHeadBatch` :

It comprises a `GPT4RecommendationBaseModel` and an LM head for language modeling.

The LM head is a linear layer mapping hidden states to vocabulary logits, and it is tied to GPT-2's token embeddings (`self.lm_head.weight = self.base_model.gpt2model.wte.weight`). That means this model predicts actual words from GPT-2's original vocabulary.

In forward pass, it takes `input_ids_prompt` and `input_ids_main` (representing the prompt and the continuation text). It does two sequential GPT-2 forward passes:

- One for the prompt: `outputs_prompt = self.base_model(input_ids_prompt, ...)` which returns hidden states and also `past_key_values` (cached attention states).
- One for the main text:
`outputs_main = self.base_model(input_ids_main, past_key_values=past_key_values, ...)` . By feeding `past_key_values` , we ensure the main text is conditioned on the prompt as if they were one continuous sequence (this is a standard technique to avoid concatenating and re-running the prompt tokens through Transformer for every step).

The output hidden states from the second pass (`outputs_main.last_hidden_state`) are then fed into `lm_head` to produce vocabulary logits. The loss is computed by comparing these logits to `labels_main` (which is essentially the same as the main text tokens, shifted by one).

The code applies a mask so that loss is only calculated on the positions corresponding to the main text (not the prompt) – because we consider the prompt as given context and only evaluate the prediction of the main text.

This model can also incorporate mutual regularization: if `regularize=True` , it expects a `collaborative_embeds` tensor to be passed in (the embeddings of the same prompt from a collaborative model) and adds an MSE loss between its own prompt embeddings and those collaborative embeddings.

During actual pretraining, however, the content model is trained first without regularization, and the reg term is mainly used when training the collaborative model.

Detailed explanation of CollaborativeGPTwithItemLMHeadBatch :

This model is used for the collaborative pretraining stage. It is structurally similar to the content model but uses an item prediction head instead of a vocabulary LM head.

The `item_head = nn.Linear(n_embd, num_items, bias=False)` is tied to the base model's `item_embeddings` weights, meaning it will produce a logit for each item ID. Its forward pass likewise takes `input_ids_prompt` and `input_ids_main` (here these would be sequences of ID tokens) and does a two-step GPT-2 forward (prompt then main, using `past_key_values`). The `outputs_main.last_hidden_state` (hidden states for each position in the item sequence) is passed through `item_head` to get `item_logits`.

Each position in `item_logits` corresponds to predicting the next item given all prior items and the prompt. The training loss is computed by shifting the logits and labels (just like a standard language model) – effectively, the model is trying to predict each item in the interaction sequence given the user and previous items.

The code subtracts `vocab_size + num_users` from the labels before computing cross-entropy so that the item IDs (which are originally large integers offset into the special token range) are mapped to a 0-based index for the softmax. Only the item positions are considered in the loss (the prompt part is excluded via attention mask similar to above).

This model also includes the regularization option: if `regularize=True`, it expects a `content_embeds` tensor (embedding of the same sequence from the content model) and computes the MSE loss to add in.

Detailed explanation of CollaborativeGPTwithItemRecommendHead :

The forward method signature is `forward(input_ids, target_ids, ...)` where `target_ids` is the multi-hot vector of target items (optional, for training).

Internally, it runs the base GPT-2 on the whole input sequence (which in fine-tuning is just the prompt ending in “will interact with”) in one go.

It then takes the final hidden state of the last token of the prompt for each batch (essentially the representation right after “will interact with”). This hidden state is fed through the item head to produce a score for each item.

After a log-softmax, the negative log-likelihood is computed as described earlier:

```
neg_ll = -mean(sum(item_log_probs * target_ids, dim=-1)) .
```

If `target_ids` is one-hot (single target), this reduces to standard cross-entropy; if it's multi-hot (multiple targets), it sums the log-probabilities of the true items.

The model returns `(neg_ll, item_log_probs)` in evaluation mode or `(neg_ll, regularize_loss, item_log_probs)` in regularization mode.

During fine-tuning training, `regularize=True` and a `content_embeds` of the same prompt (and perhaps the target sequence appended) is provided – the code concatenates the prompt and target item embeddings (`rec_embeds_prompt` and `rec_embeds_target`) and compares to the content embeddings, similar to before.